Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**Programming from First Principles**	
Topic Name	**The types of expressions may be determined by simple examination of the program text, understanding such rules, Part 1**	
Topic Code	**-**	
Unit/Module No. & Name	**9**	**Inference**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. For example, 2 + 3, x * 10, and "Hi" + name are expressions. Every expression has a type, such as integer, float, string, or boolean. Knowing the type is important because it helps the program run correctly.
2. This unit aims to explain that types are not guessed randomly. Many times, the type of an expression can be determined by simple examination of the code.
3. This means we can look at the operators, the values, and the variables and apply rules to decide the result type. Students will learn that these rules are part of type systems used in many programming languages.
4. Another objective is to help students understand why type rules matter for error detection. If the types do not match, the program may produce a type error. For example, adding a number to a string without conversion can cause problems in many languages.
5. By learning type rules, students can write safer and cleaner code. This SLM also supports exam preparation by covering common MCQ points, such as expression, operator, operand, type inference, type checking, and type error.
6. Overall, the objective is to build a strong foundation so students can understand later topics like static typing, dynamic typing, type inference, and compiler checking.

1.0 INTRODUCTION

Programming languages use types to describe what kind of data a value is. Common data types include integer, float, string, and boolean. A value like 5 is usually an integer. A value like 3.14 is usually a float. A value like "hello" is a string. A value like True is a boolean. Types matter because they decide which operations are allowed.

An expression is a part of a program that gives a value. For example, 10 + 2 is an expression that gives 12. The expression has a type. In this case, the type is integer. Another example is "A" + "B", which gives "AB". That expression has the type string. If we use an operator in a wrong way, we may get an error. For example, "A" - "B" does not make sense in most languages because subtraction is not defined for strings.

In many cases, we do not need to run the program to know the type of an expression. We can examine the code. We can look at the operator and the operands. Operands are the values or variables used with an operator. The rules of the language tell us the type. For example, in many languages, if both operands are integers and the operator is +, the result type is integer.

This idea is important in compilers and interpreters. A compiler often checks types before running the program. This is called type checking. Type checking helps find errors early. It also helps the compiler choose correct machine instructions.

So, learning how to determine the type of expressions from program text helps students write correct code and understand how programming languages work.

2.0 MEANING AND DEFINITION

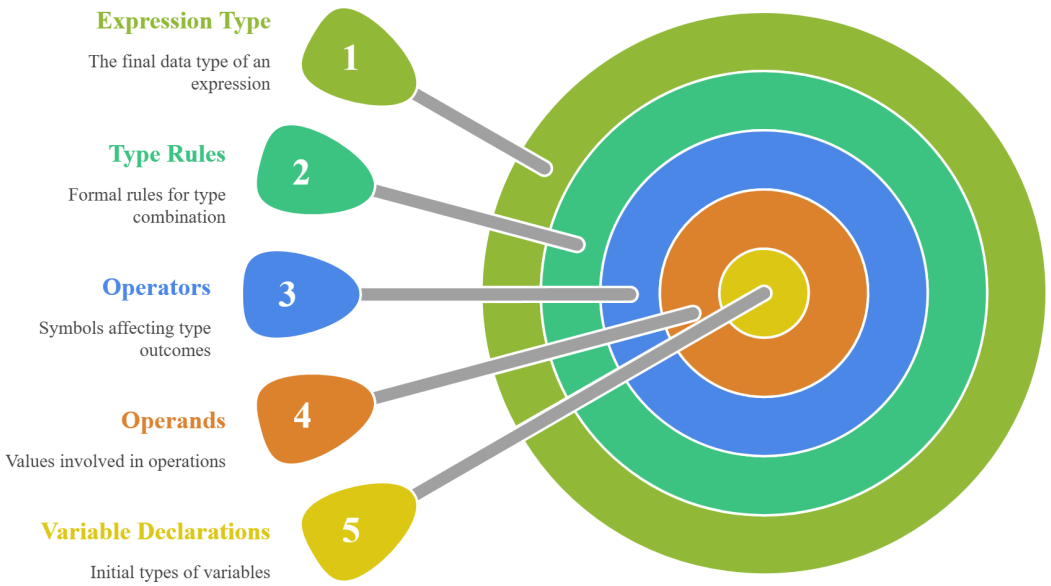
This topic focuses on understanding how expression types can be determined by rules. These rules are part of a language's type system.

Meaning:

Determining the type of an expression means deciding what kind of value the expression will produce, such as integer, float, string, or boolean, by looking at the code and applying type rules.

Fig. 1

Expression Type Determination



The image explains expression type determination, showing how type rules, operators, operands, variable declarations, and expression types interact to define the final data type of computations.

Definition:

Type determination of expressions can be defined as the process of assigning a data type to an expression by examining its structure, operators, operands, and declared variable types, using the language's typing rules.

This definition highlights that type determination uses structure and rules. It does not depend on guessing. It also shows that variable declarations can matter in many languages.

2.1 Meaning Of Expression Type

Expression type means the data type of the value that results after the expression is evaluated. For example, 4 < 9 results in True, so the expression type is boolean.

2.1.1 Role Of Operators

Operators strongly affect type. Arithmetic operators like +, -, *, and / usually work with numeric types. Comparison operators like < and == usually produce boolean results. Logical operators like and and or also produce boolean results.

2.2 Definition Of Type Rules

Type rules are formal rules that define how types combine. For example, a common rule is: integer + integer → integer. Another common rule is: integer + float → float, because the integer may be converted to float.

3.0 NATURE OR TYPE

The nature of this topic is rule-based. It shows that type determination follows fixed language rules. These rules may vary slightly between languages, but the basic idea is the same.

3.1 Type Checking Styles

3.1.1 Static Type Checking (Basic Idea)

Static type checking means checking types before the program runs. This happens in compiled languages like C, C++, and Java. The compiler checks whether expressions have correct types based on rules.

3.1.2 Dynamic Type Checking (Basic Idea)

Dynamic type checking means types are checked while the program runs. This happens in languages like Python. Even in dynamic typing, expressions still follow type rules, but errors may appear at runtime.

3.2 Type Inference (Basic Idea)

3.2.1 What Type Inference Means

Type inference means the language can guess the type of a variable or expression from context. For example, if x = 10, many systems infer that x is an integer.

3.2.2 Why Rules Are Needed

Type inference still uses rules. The system checks operators and operands to infer types. So, rules remain the base of type determination.

3.3 Common Rule Patterns

3.3.1 Arithmetic Rule Pattern

If both operands are integers, integer operators often give integer results. If any operand is float, the result often becomes float.

3.3.2 Comparison Rule Pattern

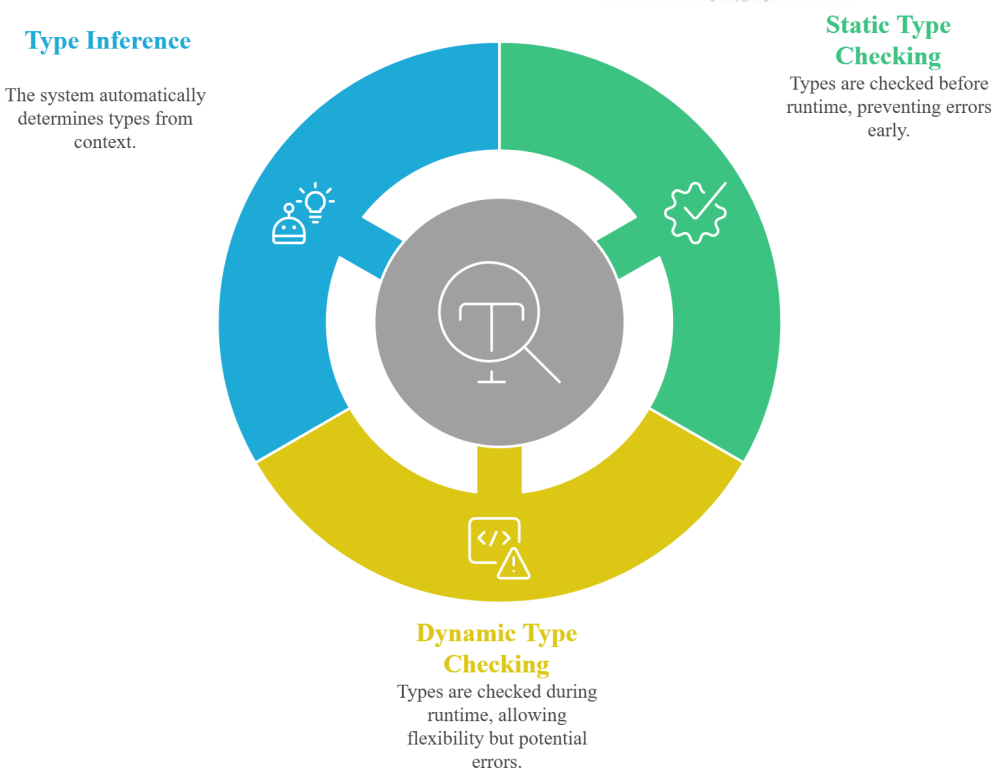
Comparison operators usually return boolean values.

3.3.3 String Rule Pattern

In many languages, string concatenation uses + but only when both operands are strings or can be converted safely.

Fig.2

Type Checking Styles



The image illustrates type checking styles, comparing static type checking, dynamic type checking, and type inference, highlighting when types are verified and how errors are prevented or detected.

4.0 EXAMPLES AND CASE STUDIES

Examples help students learn type rules by seeing them in code-like form.

4.1 Example: Arithmetic Expressions

Consider these expressions:

- 2 + 3 → both operands are integers, so type is integer.
- 2 + 3.5 → one operand is float, so result type is float.
- 10 / 2 → in many languages, division may produce float, even if operands are integers.

This shows that the operator also matters, not only the operand types.

4.2 Example: Comparison Expressions

Consider:

- 5 > 2 → result is True/False, so type is boolean.
- x == y → result is boolean, even if x and y are integers or strings.

This shows that comparison gives boolean type.

4.3 Example: String Expressions

Consider:

- "Hi" + "All" → result is string, because it joins strings.
- "Age: " + 20 → in many languages this is a type error unless 20 is converted to string.
- "Age: " + str(20) → result is string.

This shows how type rules prevent unsafe mixing.

4.4 Case Study: Finding Errors Before Running

In a statically typed language, the compiler may stop this code:

- int x = 5;
- x = "hello";

The compiler checks types and sees mismatch. It reports an error before running. This saves time and prevents runtime failure.

In a dynamically typed language, a similar error may appear during execution. So students must understand types in both styles.

1. Expression types can often be found by reading the program text.
2. Operators and operands decide the type of an expression.
3. Type rules define how types combine in operations.
4. Arithmetic operators mostly work with numeric types.
5. Comparison operators usually produce boolean values.
6. String concatenation may require both operands to be strings or convertible.
7. Static type checking finds errors before running the program.
8. Dynamic type checking may find errors during program execution.
9. Type inference uses rules to guess types from context.
10. Learning type rules helps students avoid common programming mistakes.

5.0 KEYWORDS AND TOUCH VOCABULARY

Keyword	Meaning
Expression	Code that produces a value
Type	Category of data like int, float, string, boolean
Operand	Value or variable used with an operator
Operator	Symbol like +, -, *, /, ==, <
Type Rule	Rule that decides result type from operand types
Type Checking	Verifying types are used correctly
Static Typing	Type checking before program runs
Dynamic Typing	Type checking during program execution
Type Inference	Determining type from context automatically
Type Error	Error caused by incompatible types
Boolean	True/False data type
Concatenation	Joining strings together